Chapter2: Sequential File Structures

Chapter Outline

- **Blocks organization (Global block organization):**

  Contiguous Blocks vs Linked Blocks

- **Records internal format (Internal block organization):**

  Fixed length vs Variable length records

- **Taxonomy of Simple Sequential File Structures:**

  TOF , TOVS , TOV$\overline{S}$, T$\overline{O}$F , T$\overline{O}$VS , T$\overline{O}$V$\overline{S}$, LOF , LOVS , LOV$\overline{S}$, L$\overline{O}$F , L$\overline{O}$VS , L$\overline{O}$V$\overline{S}$

- **Case study : TOF**

  Insertion, Deletion, Bulk Loading, Reorganization, Merging two TOF files, …

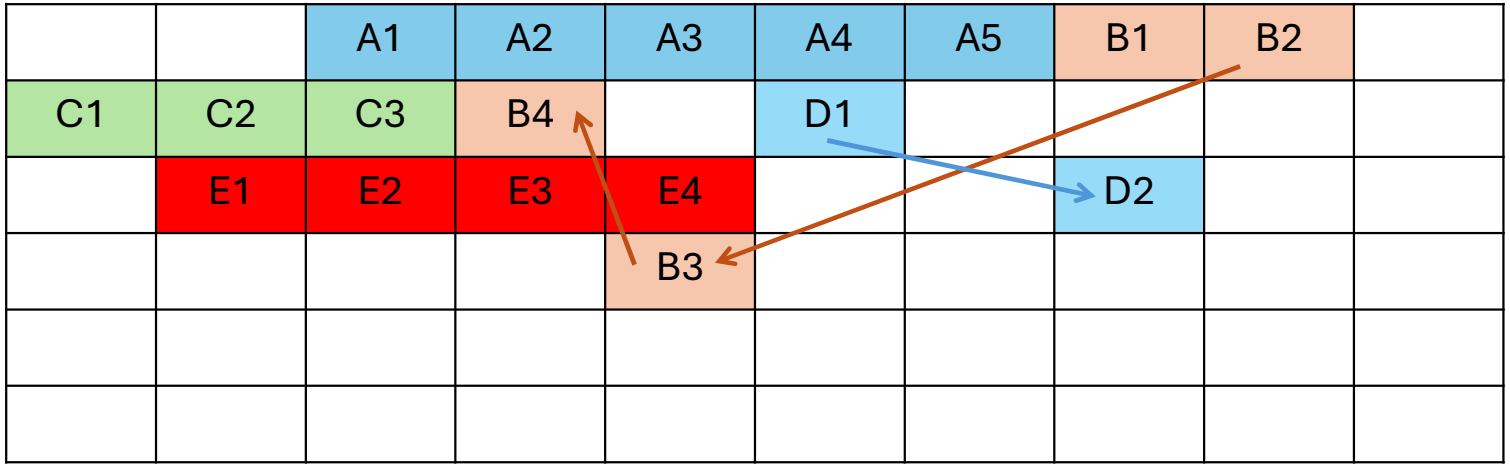
Blocks organization

→ In sequential structures, the blocks forming the file can be organized according to two approaches: contiguous blocks and chained blocks.

Examples :

→ Contiguous blocks (e.g. Files A (5 blocks), C (3 blocks) and E (4 blocks))

→ Chained blocks (e.g. Files B (4 blocks) and D (2 blocks))

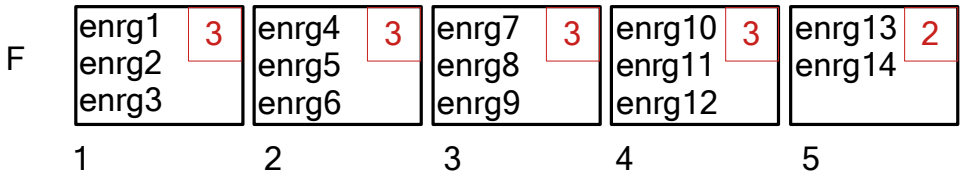


Blocks organization

→ In sequential structures, the blocks forming the file can be organized according to two approaches: contiguous blocks and chained blocks.

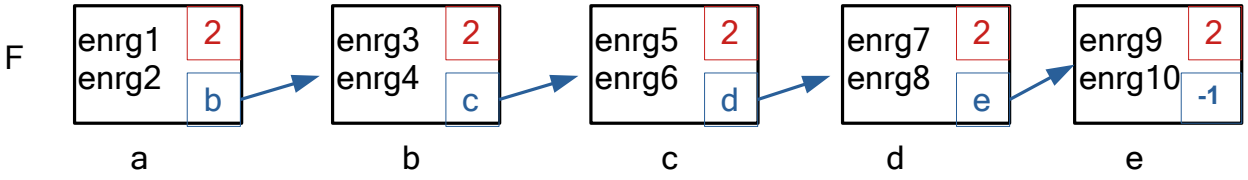
a) Contiguous (the file is "seen" as a **Table of Blocks** in SM)

```
Type blocT = Struct
        tab : array[ 3 ] of typeRec
        NB : int
    end_struct
Var
    F : FILE of blocT BUFFER … .  HEADER ( …  )
```



b) Chained (the file is "seen" as a **List of Blocks** in SM)

```
Type blocL = Struct
        tab : array[ 2 ] of typeRec
        NB : int
        link : int
    end_struct
Var
    F : FILE of blocL BUFFER … .  HEADER ( …  )
```

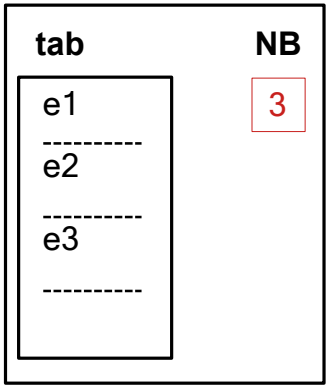


Records internal format - Internal block organization

→ Depending on the need, the records can be of **fixed sizes** or **variable sizes**.

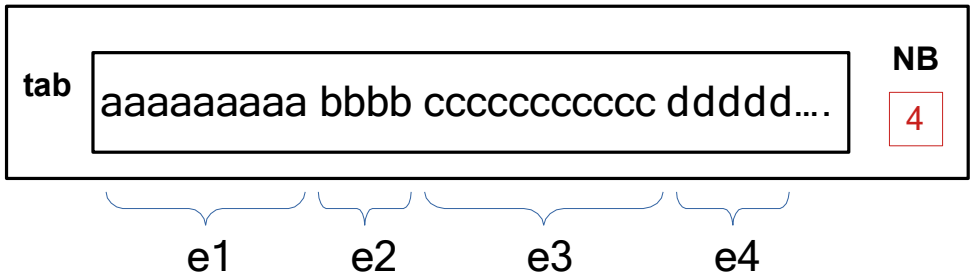
1. For **fixed size records:**

   Each block will contain an *array of records*



2. For **variable size records :**

   Each block will contain an *array of bytes (characters)*
   Records are placed as strings (of variable length) inside the byte array



Records internal format - Internal block organization

→ **An example of fixed-length records :**

A file storing **employee** data where each record has a consistent structure with fields of predetermined lengths.

Record Structure:

- *Employee ID: 5 characters*
- *Last name: 25 characters*
- *First name: 25 characters*
- *Department: 10 characters*
- *Salary: 6 characters*

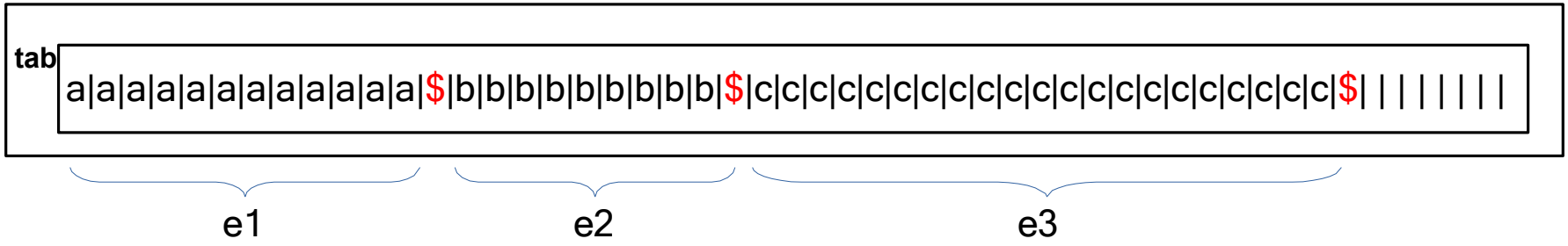
Example data:

00121	Boulakradeche	Mohamed	Finance	090000
00122	Hidouci	Khaled Walid	IT	250000
00123	Kermi	Adel	HR	085000

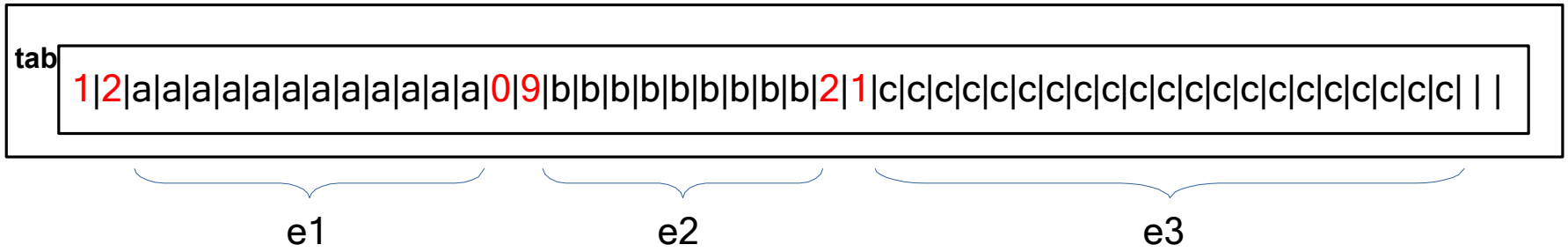
Records internal format - Internal block organization

→ **Variable Size Records :**

Records can be separated by a special character, e.g.: '$'



Records can be preceded by their size



Fields of variable length (within records) can also be separated using the same approaches (either separated by a special character or preceded by their size).

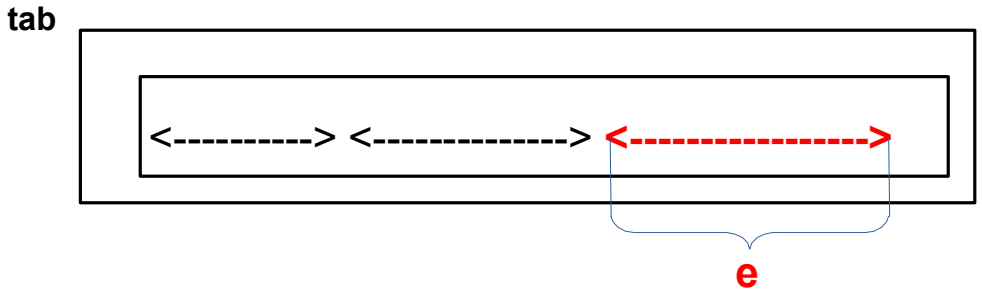
Records internal format - Internal block organization

→ Variable Size Records : with or without overlap between blocks

In the case of variable-size records, when inserting a new record ***e*** into a block, if the available space is not sufficient to fully contain string ***e***, there are 2 possibilities:

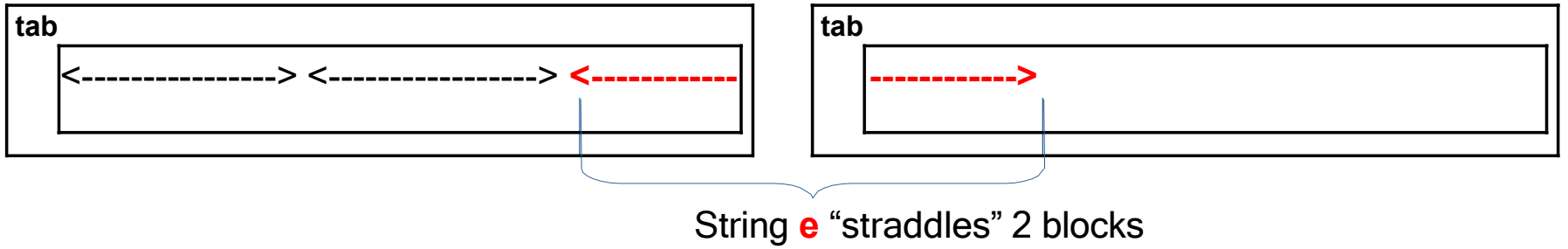
- **Non-overlapping scheme**

The remaining empty space is not used and string ***e*** is inserted into another block containing sufficient empty space.



- **Overlapping scheme**

All available empty space in the current block is used to insert the first bytes of string ***e***. The remainder is inserted into the next block(s)



Records internal format - Internal block organization

→ **An example of variable-length records :**

A file storing ***customer feedback***, where each record includes a customer ID, customer last name, customer first name, and a comment of varying length.

Record Structure:

- *Customer ID: Varies (but typically fixed for simplicity, e.g. 3 characters)*
- *Last name: Varies*
- *First name: Varies*
- *Comment: Varies*

Example data:

```
001Laskri|Mohamed Tayeb|Excellent service!
002El-Alia|Nadia|I am not satisfied with the product.
003Kermi|Adel|Great quality, but delivery was slow.
```

Taxonomy of Simple Sequential File Structures

→ A sequential file structure can be defined by choosing from the following possibilities:

1. The blocks are **contiguous**, i.e. the file is seen as a **Table** (***T***) or
   The blocks are **chained**, i.e. the file is seen as a **List** (***L***)

2. The file is **Ordered** (***O***) according to the key field or
   The file is **not ordered** (***$\overline{O}$***)

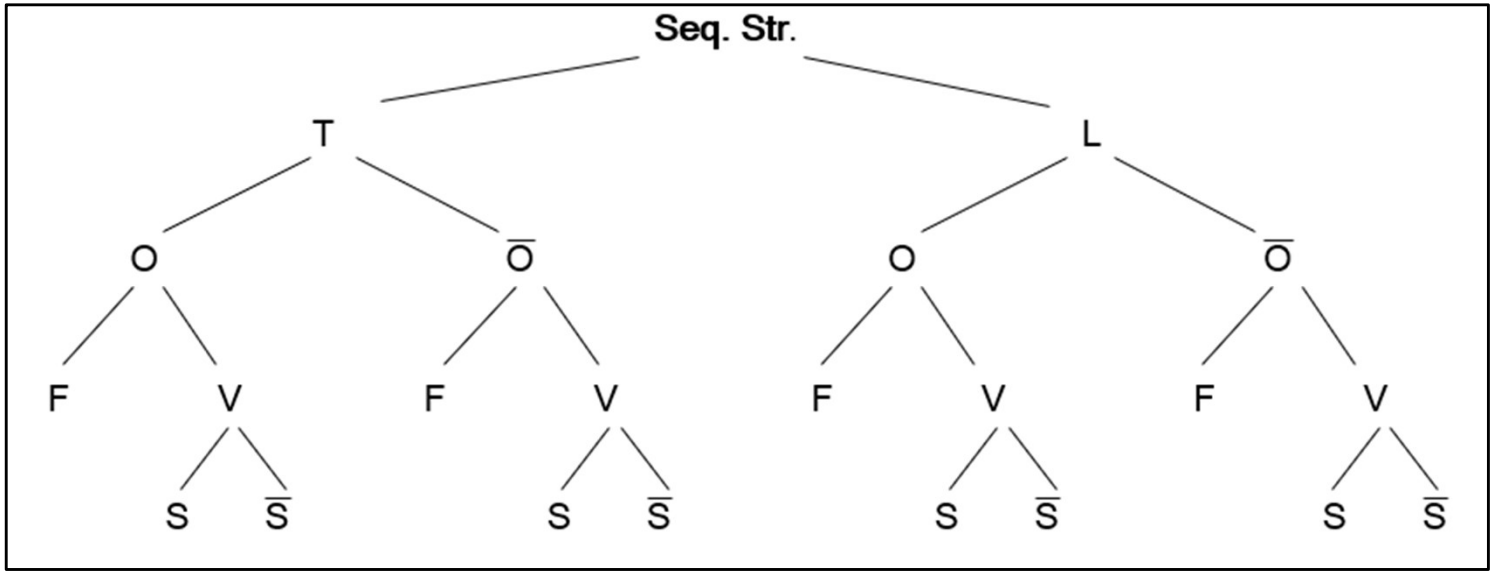
3. Records inside blocks occupy **Fixed** (***F***) or **Variable** (***V***) sizes

4. In the case of variable record sizes, storage "**Straddling**" several blocks can be authorized (***S***) or not (***$\overline{S}$***).

Taxonomy of Simple Sequential File Structures

1- File "seen" as a table (**T**) or as a Liste (**L**)
2- Ordered file (**O**) or Not ordred ($\overline{\mathbf{O}}$)
3- Records of Fixed size (**F**) or Variable size (**V**)
4- In the case of variable sizes (V), records can “Straddle” several blocks (***S***) or not ($\overline{\boldsymbol{S}}$).

→ Combining all these possibilities, we can define 12 simple methods for sequential file structures :



→ Each leaf node represents a simple method for a sequential file structure :
TOF , TOVS , TOV$\overline{S}$, T$\overline{O}$F , T$\overline{O}$VS , T$\overline{O}$V$\overline{S}$, LOF , LOVS , LOV$\overline{S}$, L$\overline{O}$F , L$\overline{O}$VS , L$\overline{O}$V$\overline{S}$

An overview of the TŌF method

- The **TŌF** method is probably the simplest of the 12 sequential methods.
  Blocks are contiguous (like a table). Records of fixed size are not ordered.

→ Here is a possible solution to design the method:

  - **Searching** for a record consists of sequentially traversing the file, block by block, until it is found or until the last block is reached.
  - **Insertion** of a new record ***e*** is done directly in the last block if it is not full, otherwise we add a new block.
  - **Deleting** a record involves moving the last record in the file to overwrite the record to be deleted (this avoids gaps in the file blocks).

- All blocks of the file will thus be 100% full, except possibly the last one.
- There is no need for periodic reorganization.

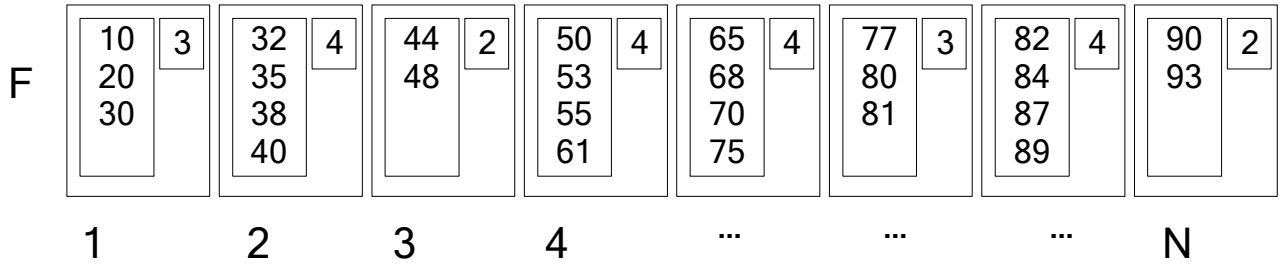
```
Type Tblock = struct
        tab : array[ b ] de TypeRecord
        NB : int
      end_struct
Var
    F : FILE of Tblock BUFFER buf1, buf2 HEADER( nblk:int )
                                        /* number of blocks in F */
```

Case study: TOF

Ordered files formed by contiguous blocks (i.e. "TO*" methods except TOVS) are interesting, because searching records can be fast (using the external binary search algorithm ***O(Log n)***).

Their major disadvantage lies in the insertion operation which can become very costly in I/O (because of inter-block shifts necessary to maintain order)

As an example, here is a possible solution to design the method **TOF** :




```
Type Tblock = struct
                tab : array[ b ] of Trec
                NB : int
              end_struct
```

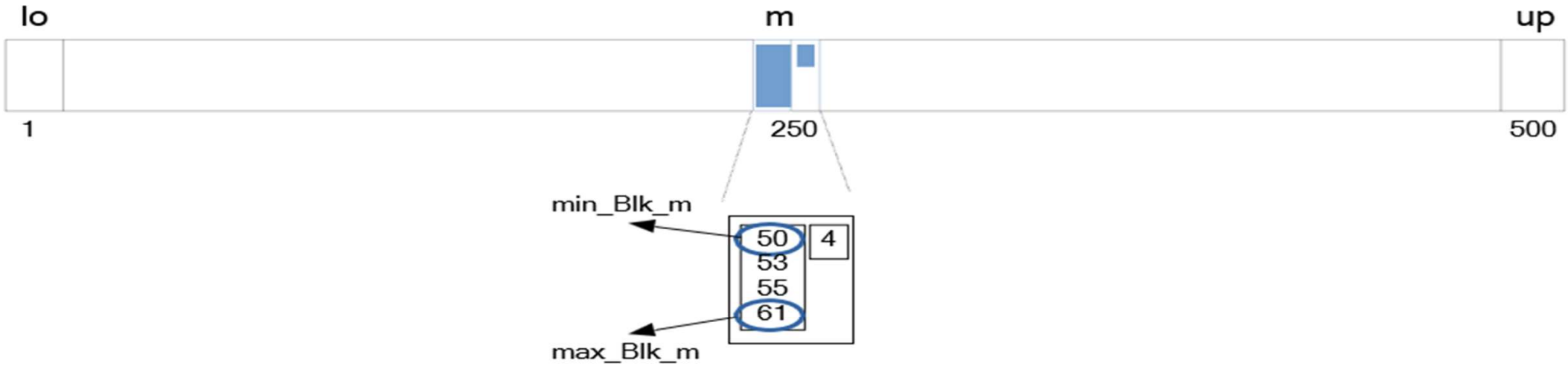
```
Type Trec = struct
              key:anytype
              …
            end_struct
```

F : FILE of Tblock BUFFER buf1, buf2 HEADER(nblk:int , nrec:int)

To mitigate the disadvantage of inserts, it is best if the blocks are not full ⇒ e.g. the average file loading factor » 70%

Case study: TOF → External Binary Search

To search for v in the file (the block area [lo .. up], where lo=1 and up=N), you must:
- search for v locally in the middle of the area (the block number m = (lo + up) / 2)
- if not found, continue searching in one of the two halves of the initial area: [lo .. middle-1] or [middle+1 .. up] (according to the comparison of v < min_Blk_m or v > max_Blk_m)

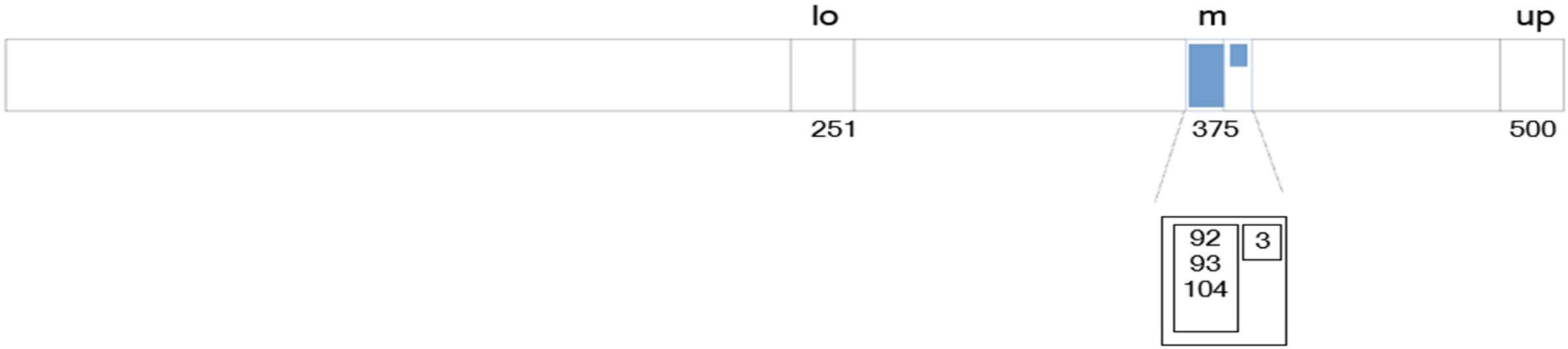


Example : Searching v = **75** in area [lo=1 .. up = 500]
⇒ m ← (lo+up) div 2 = 250
⇒ **readBlock (F , 250 , buf)**
⇒ internal_search_in_buf
⇒ lo ← m+1 = 251

Case study: TOF → External Binary Search

To search for v in the file (the block area [lo .. up]), you must:

- search for v locally in the middle of the area (the block m = (lo + up) / 2)
- if not found, continue searching in one of the two halves of the initial area: [lo .. middle-1] or [middle+1 .. up] (according to the comparison of v < min_Blk_m or v > max_Blk_m)



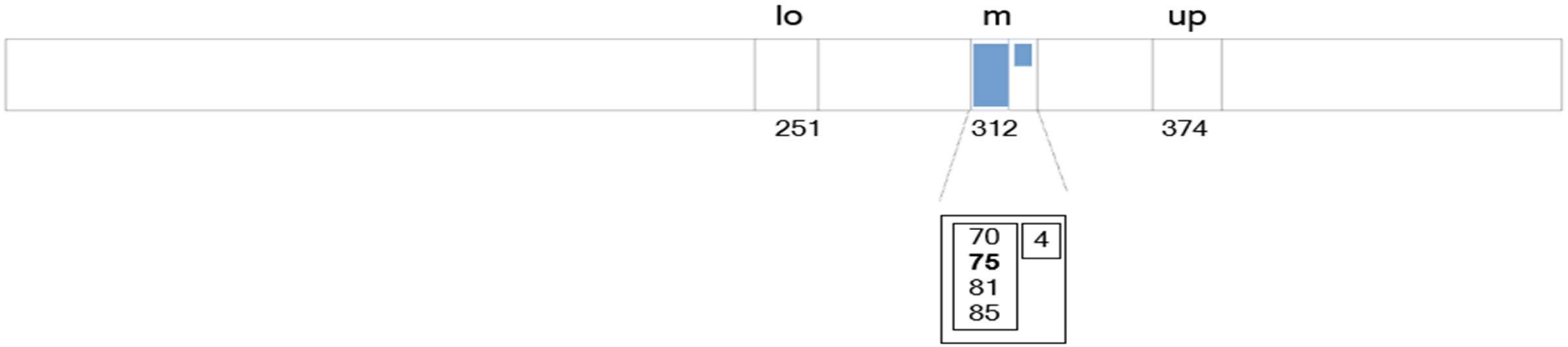
Example : Searching v = **75** in area [lo=251 .. up = 500]

⇒ m ← (lo+up) div 2 = 375
⇒ **readBlock (F , 375 , buf)**
⇒ internal_search_in_buf
⇒ up ← m-1 = 374

Case study: TOF → External Binary Search

To search for v in the file (the block area [lo .. up]), you must:

- search for v locally in the middle of the area (the block m = (lo + up) / 2)
- if not found, continue searching in one of the two halves of the initial area: [lo .. middle-1] or [middle+1 .. up] (according to the comparison of v < min_Blk_m or v > max_Blk_m)



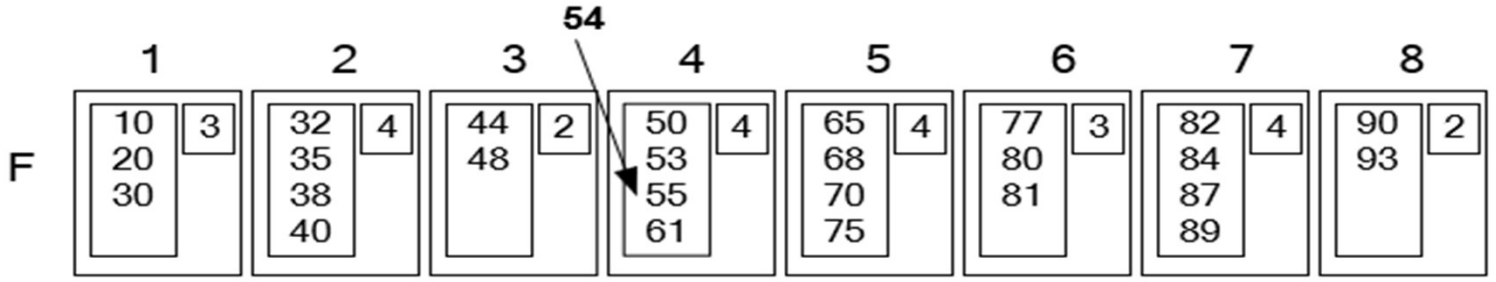
Example : Searching v = **75** in area [lo=251 .. up = 374]

⇒m ← (lo+up) div 2 = 312
⇒**readBlock (F , 312 , buf)**
⇒internal_search_in_buf
⇒STOP searching … [75 found in block 312 at position 2]

Case study: TOF → Insertion Mechanism

To insert record *e* = <*v*,...> in *F* :
1- Search for *v* in *F* ⇒results : *found*:bool , *numblk*:int , *pos*:int
2- IF (not *found*)
    1. - insert *e* into block *numblk* at position pos, shifting one position to the right, all records > *e*
    2. - IF the current bloc (*numblk*) was full before insertion of *e*, the last record in the block is ejected. Shifting must then continue in the next block (*numblk*+1) :
        1. - let *e* be the last record ejected from current block
        2. - *numblk* ← *numblk*+1 ; *pos* ← 1 ; GOTO 2.1
3- Stop

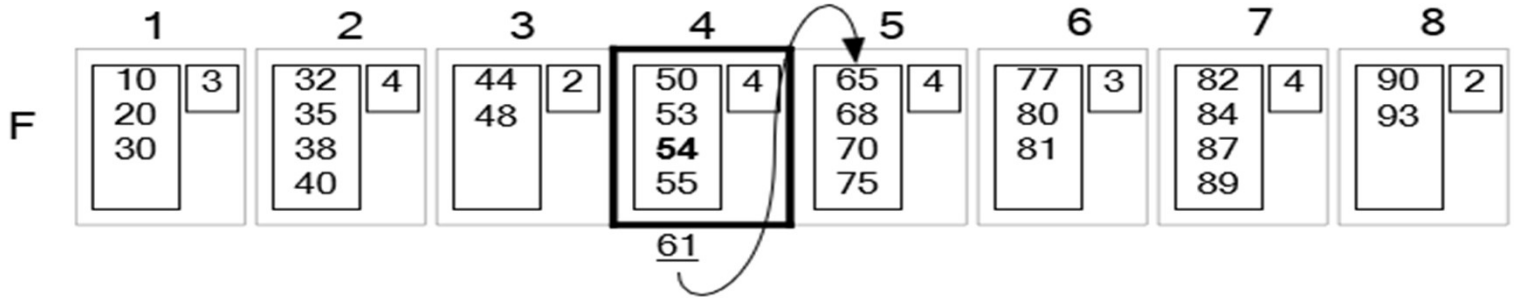
Example : insert record : < **54**…. >
⇒shifting starts at block 4, pos 3



Case study: TOF → Insertion Mechanism

To insert record *e* = <*v*,...> in *F* :
1- Search for *v* in *F* ⇒results : *found*:bool , *numblk*:int , *pos*:int
2- IF (not *found*)
    1. - insert *e* into block *numblk* at position pos, shifting one position to the right, all records > *e*
    2. - IF the current bloc (*numblk*) was full before insertion of *e*, the last record in the block is ejected. Shifting must then continue in the next block (*numblk*+1) :
        1. - let *e* be the last record ejected from current block
        2. - *numblk* ← *numblk*+1 ; *pos* ← 1 ; GOTO 2.1
3- Stop

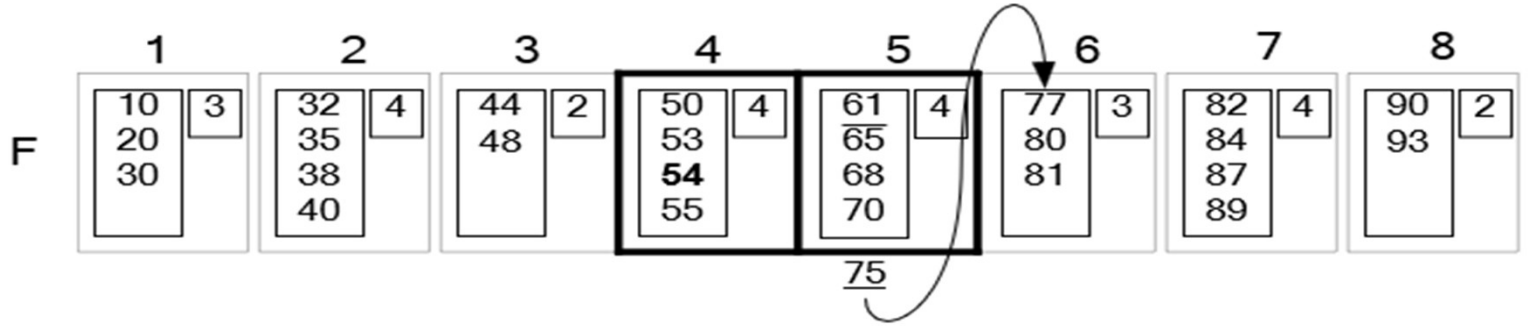
Example : insert record : < **54**…. >
⇒the last record < 61… . > is ejected from block 4
⇒< 61… . > must be inserted in block 5 at pos 1



Case study: TOF → Insertion Mechanism

To insert record *e* = <*v*,...> in *F* :
1- Search for *v* in *F* ⇒results : *found*:bool , *numblk*:int , *pos*:int
2- IF (not *found*)
    1. - insert *e* into block *numblk* at position pos, shifting one position to the right, all records > *e*
    2. - IF the current bloc (*numblk*) was full before insertion of *e*, the last record in the block is ejected. Shifting must then continue in the next block (*numblk*+1) :
        1. - let *e* be the last record ejected from current block
        2. - *numblk* ← *numblk*+1 ; *pos* ← 1 ; GOTO 2.1
3- Stop

Example : insert record : < 54…. >
⇒the last record of block 5 (record < 75…. >) is ejected
⇒< 75… . > must be inserted in block 6 at pos 1

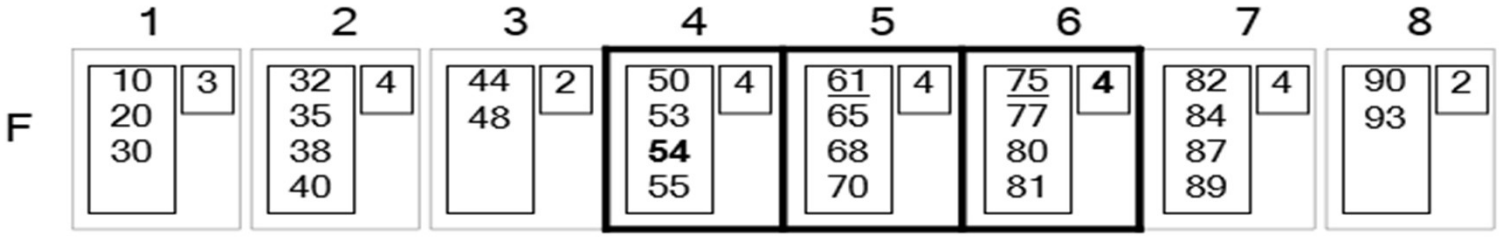


Case study: TOF → Insertion Mechanism

To insert record *e* = <*v*,...> in *F* :
1- Search for *v* in *F* ⇒results : *found*:bool , *numblk*:int , *pos*:int
2- IF (not *found*)
1. - insert *e* into block *numblk* at position pos, shifting one position to the right, all records > *e*
2. - IF the current bloc (*numblk*) was full before insertion of *e*, the last record in the block is ejected. Shifting must then continue in the next block (*numblk*+1) :
    1. - let *e* be the last record ejected from current block
    2. - *numblk* ← *numblk*+1 ; *pos* ← 1 ; GOTO 2.1

3- Stop

Example : insert record : < 54…. >
⇒block 6 was not full, therefore no more inter-block shifts needed



TOF → Cost of insertion algorithm using inter-block shifts

In the best case, an insertion is a binary search + a write of the last block visited by the search. This happens when the block where the record should be inserted is not full. So the I/O cost in this case is ***O(Log n)***, where n is the number of blocks in the file

In the worst case, an insert requires a binary search operation followed by n inter-block shifts. This can happen when, for example, all the blocks are 100% full and the insertion must be done at the beginning of the file. So the I/O cost in this case is ***O(n)***

The average cost varies between these 2 limits depending on the loading of the file. The higher the loading factor, the closer the insertion cost is to O(n). The lower it is, the closer the cost is to O(Log n).
The loading factor increases with the number of insertions. As a result, the average cost is in practice closer to the unfavorable case (***O(n)***)

The solution is to periodically perform complete (and costly) **reorganizations** of the entire file (possibly adding new blocks) to bring down the loading factor and obtain (temporarily) acceptable insertion costs

Case study: TOF → Deletion

Deletion in a TOF file can be physical (using inter-block shifts) and costly as insertions.

But since it is planned to carry out reorganization operations on a regular basis (in order to avoid performance degradation due to insertions), it is then preferable to opt for logical deletions (much simpler and faster)

Logical deletions

We add an array of booleans (in the block structure) to indicate the status of existing records ("deleted" or "not-deleted")

```
Type Tblock = struct
                tab      : array[ b ] of typeRec
                erased   : array[ b ] of bool
                NB       : int
              end_struct
```

To logically delete a record with ***v*** as the key value :

```
1. search( F , v ) → [ found , numBlk , pos ]
   // at the end of the binary search, global variable buf contains the last block visited
2. IF ( found )
              buf.erased[ pos ] = true      // so there is no need to read the block a second time
              writeBlock( F , numBlk, buf )
   End_If
```

Case study: TOF → Bulk Loading

Sometimes we have, from the start, a set of records to construct a new TOF file Instead of using the insert algorithm repeatedly, it is more efficient to insert all initial records sorted and grouped to optimize I/O operations during this construction phase and to obtain at the end of the build, a weakly loaded file. This operation of building the file with initial records is called **Bulk Loading**

```
open( F, "myfile.dat", 'N' )
i ← 1; j ← 1          // block num to fill and record num in the block
FOR k = 1 , n
    read( e )
    IF  ( j ≤  0.5 * b )     // to obtain a loading factor = 50%
        buf.tab[ j ] ← e; buf.erase[ j ] ← false; j++
    ELSE
        buf.NB ← j-1; writeBlock( F , i , buf ) ; i++
        buf.tab[ 1 ] ← e; buf.erase[ 1 ] ← false; j ← 2
    End_If
End_For
// at the end of the loop, the last block is written
buf.NB ← j-1; writeBlock( F, i, buf )
// setup headers (block and record count)
setHeader( F, "nblk", i ); setHeader( F, "nrec", n );
close( F )
```

Case study: TOF → Reorganization

When the file loading factor becomes too large the overall performance of the method degrades significantly. The solution is to completely reorganize the file
A simple way to perform this **reorganization** operation is to perform a bulk loading to construct a new file using the undeleted records from the old file

```
open( F1, "oldFile.dat", 'E' ) ; N1 ← getHeader(F1,"nblk") ; nr ← 0
open( F2, "newFile.dat", 'N' ) ; i2 ← 1; j2 ← 1       // block num  and record num  (F2)
FOR i1 = 1 , N1
    readBlock( F1 , i1 , buf1 )
    FOR j1=1 , buf1.NB
       IF ( not buf1.erase[ j1 ] )
           nr++       // number of records to insert in F2
           IF  ( j2 ≤  0.7 * b ) // to obtain a loading factor = 70%
              buf2.tab[ j2 ] ← buf1.tab[ j1 ]; buf2.erase[ j2 ] ← false; j2++
           ELSE
              buf2.NB ← j2 - 1; writeBlock( F2 , i2 , buf2 ) ; i2++
              buf2.tab[ 1 ] ← buf1.tab[ j1 ]; buf2.erase[ 1 ] ← false; j2 ← 2
           End_If
        End_If
    End_For
End_For
// write last block of F2 and set header ...
buf2.NB ← j2 - 1; writeBlock( F2, i2, buf2 )
setHeader( F2, "nblk", i2 ); setHeader( F2, "nrec", nr );
close( F1 ) ; close( F2 )
```

Case study: TOF → Merging two TOF files

Merging 2 ordered files produces one ordered file containing all the records from the first two files.

F1

1	2	3	4	5	6
10 20 30 | 3	32 35 38 40 | 4	44 48 | 2	50 53 55 | 3	65 68 70 | 3	77 80 85 | 3

F2

1	2	3	4	5
5 15 20 25 | 4	33 45 | 2	47 48 49 50 | 4	50 58 60 | 3	81 90 | 2

F3

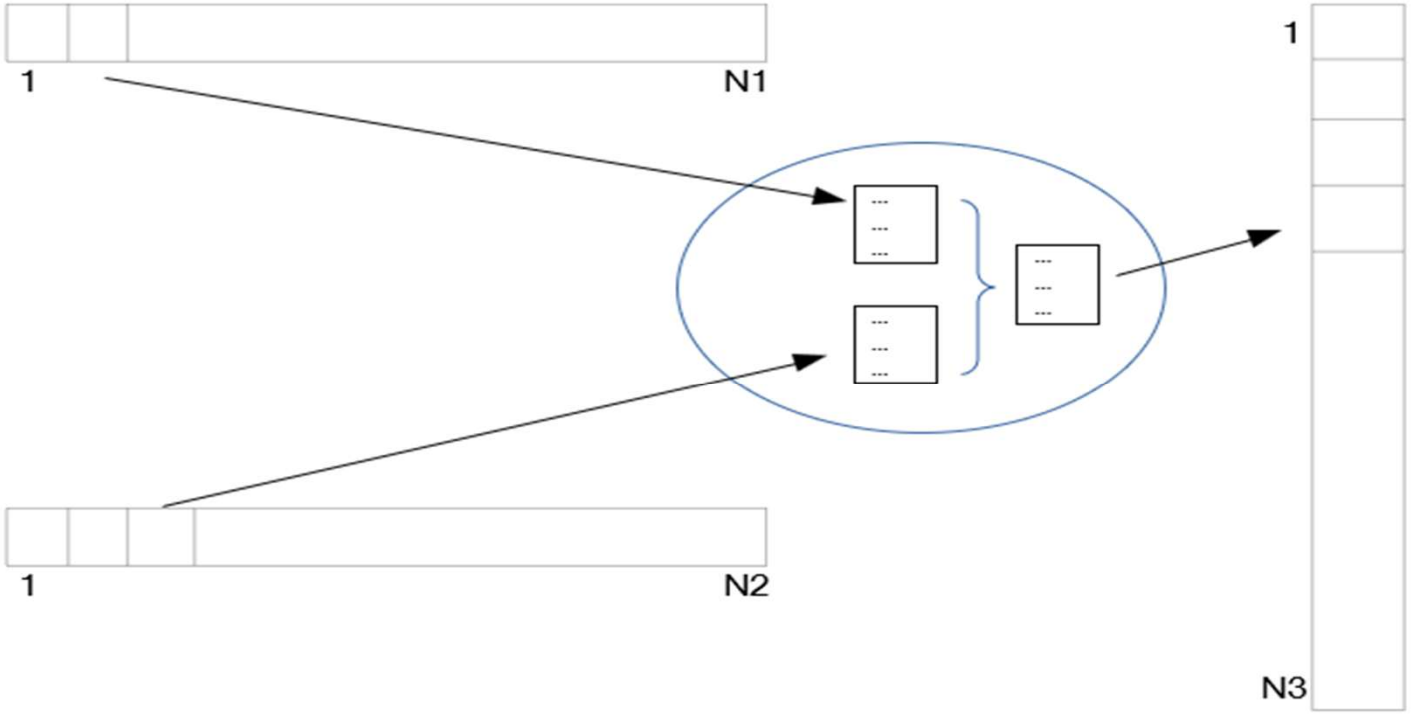
1	2	3	4	5	6	7	8	9	10	11
5 10 15 | 3	20 25 30 | 3	32 33 35 | 3	38 40 44 | 3	45 47 48 | 3	48 49 50 | 3	50 50 53 | 3	55 58 60 | 3	65 68 70 | 3	77 80 81 | 3	85 90 | 2

In this example, duplicate values are accepted and the chosen loading factor = 75%

Case study: TOF → Merging two TOF files

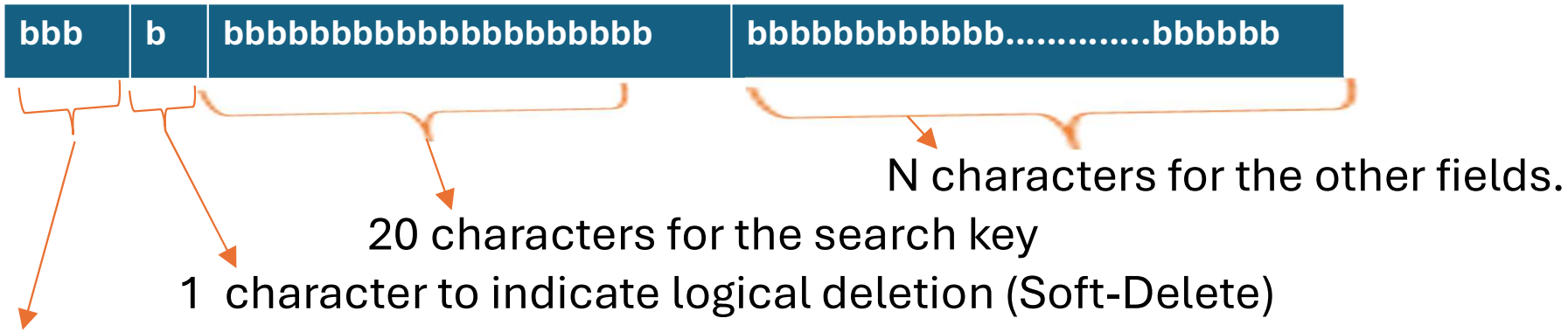
To carry out this operation efficiently, 3 buffers are used (one for each file).

We sequentially go through the two files to be merged at the same time (with 2 buffers) in order to process all the records of the two files in ascending order.
This ordered traversal of all the records makes it possible to carry out Bulk-Loading to construct the result file (with the 3rd buffer)



Case study: LnOVS

- The record may take the following form:



1- Declaration of structures :

```
TYPE  Tbloc = STRUCTURE
          Tab : Array [1 .. b] de characters ;
          link: entier;
END;
VAR  F  FICHIER de Tbloc BUFFER  Buf  ENTETE (entier, entier, entier, entier )
                                      ❶     ❷     ❸     ❹
```

❶ : Number of the first block (Head) - ❷ : Number of the last block (Queue)
❸ : The first available position in the last block.
❹ : The number of characters lost due to logical deletions.

```
Procedure Search_LnOVS (FileName: char; Key: char[20];  VAR Found : Boolean; VAR i, j : integer)
VAR ……..
BEGIN
  Open (F, FileName, 'E');   i:= GetHeader(F, 1) ;  j:= 1;   Found := False ;  ReadBlock(F, i, Buf);
  WHILE (NOT Found) and (i <> GetHeader(F, 2) or (i = GetHeader(F, 2) and j<> GetHeader(F, 3)) DO
   BeginW
        retrieve_string(3, i, j, length);  retrieve_string(1, i, j, delete);  retrieve_string(20, i, j, search_key);
        IF (search_key= key) and (delete = 'N') THEN Found := True
        ELSE
              j := j + number(length) – 20;
              IF (j > b) THEN
                    j := j – b;  i := Buf.link;  ReadBlock(F, i, Buf);
              EndIF;
        EndIF;
  EndW;
  Close(F);
END;
```

```
Procedure retrieve_string (n : integer ;  VAR i, j : integer ; VAR ch : char)
VAR k : integer;
Begin
  FOR  k := 1, n  DO
  BeginF
        IF (j <= b) THEN
              ch[k] := Buf.tab[j];
              j := j + 1
        ELSE // Overlapping or Straddling
              i := Buf.Link ;
              ReadBlock(F, i, Buf);
              ch[k] := Buf.tab[1];
              j := 2;
        EndIF;
  EndF;
End;
```

```
Procedure Insert_LoNVS (FileName: char ; Key: char[20]; Record: char[... ]);
VAR .....
BEGIN
  Search_LnOVS (FileName, Key, Found, i, j);
  IF (Found = TRUE) THEN Write (« Impossible to insert – The key already exists »)
  ELSE
    Open (F, FileName, 'E');  i:= GetHeader(F, 2) ; j:= GetHeader(F, 3) ;
    string_length := str (length(record), 3) ; // convert the length to a string of 3 characters.
    Write_string (3, string_length, i, j); // Copy string_length into Buf starting at position j
    Write_string (1, 'N', i, j);
    Write_string (Length(record), record, i, j);
    Buf.link := -1;
    WriteBlock (F, i, Buf);
    IF (i <> GetHeader (F, 2) THEN
          SetHeader(F, 2, i);
    EndIF;
    SetHeader(F, 3, j);
    Close(F);
  EndIF;
END;
```

```
Procedure Write_string (n : integer ;  ch: char ; VAR i, j : integer )
VAR k : integer; i1 : integer ; ....
Begin
   FOR  k := 1, n  DO
   BeginF
         IF (j <= b) THEN
                Buf.tab[j] := ch[k];
                j := j + 1
         ELSE // Overlapping or Staddling
                i1 := AllocBlock(F);
                Buf.link := i1;
                WriteBlock(F, i, Buf);       i := i1;
                Buf.tab[1] := ch[k];
                j := 2;
         EndIF;
   EndF;
END;
```

Case study: TOF → Other operations

- **Guided Exercises n°2 :**
  - Objectives: Writing algorithms and programming in C, using the abstract machine model discussed during the course session, to perform operations on sequential file structures …